The law of double negation fails: no type-theoretic function maps every doubly negated type to the type itself

The Univalent Foundations Program

hott-book:logic.tex:local-snapshot-20261012; CC BY-SA 3.0 Unported

Editorial note. The counted statement is the author's thm environment at source lines 177-180 and its proof environment opens on line 181, the next line; the pairing is by adjacency and the retained proof range 181-221 is the whole environment, which ends with the derivation of an element of \emptyt.

Editorial note. The retained counter values (chapter 3, section 2, theorem offset 1) make the counted theorem print as Theorem 3.2.2, matching the original book numbering read from the label-number dump of the pinned snapshot (newlabel thm:not-dneg = 3.2.2).

Editorial note. No context span is retained: every label the statement and the proof use inside the retained range is defined there (the two equations eq:fpaut and eq:fpfaut are labelled inside the proof).

Editorial note. Four labels the proof cites lie outside the card and are declared in external_references as non-load-bearing navigation pointers: thm:type-is-not-a-set (book Example 3.1.9, its defining line 133 of the same file), eq:transport-arrow (2.9.4), sec:compute-universe (2.10) and rmk:true-neq-false (2.12.6). The last three are results of chapter 2, whose file is not part of the pinned corpus, so their pointer spans are the label lines of this chapter's label-number dump; nothing load-bearing is replaced by any of them.

Editorial note. The retained preamble is the book's own macro library and layout adapter as stored for this chapter (retained-preamble.tex, the file's full length read at generation time); no package outside it is requested.

Editorial scope. One original result of the pinned HoTT Book source tree, counted once: the failure of the law of double negation (author label thm:not-dneg, book Theorem 3.2.2).

Allowed standard prerequisites (editorial).

- Negation as a function type. Negation is not a primitive: \neg A is by definition the function type A \to \emptyt, so a proof of \neg\neg A is a function (A \to \emptyt) \to \emptyt.. It is the definition the proof starts from (source line 182).

- Univalence and transport along a path of types. A path p : A = B of types induces an equivalence of the types, and dependent functions act on paths: apd f p transports f(A) along the family.. The proof applies the action on paths of a putative f at the path ua(e) of Bool with itself (source lines 190-196).
- Fixed-point-free autoequivalence of the booleans. The swap e : Bool \simeq Bool sending true to false and false to true has no fixed point.. The contradiction is extracted by applying a fixed-point equation to e (source line 216).
- Function extensionality on a negated type. Any two elements of \neg\neg Bool are equal, because \neg\neg Bool is a function type into \emptyt and \emptyt is a proposition.. It collapses the transported argument to the original one (source lines 200-204).

Source reference. Original remark 3.1.9: sources/hott-logic/pinned-original-source.tex, source lines 133–133. The book's Example 3.1.9 (eg environment) which introduces the fixed-point-free swap of Bool used at source line 189. Navigation pointer only: the counted proof is retained in full and this result is cited for its construction, not for its conclusion.

Source reference. Original equation 2.9.4: sources/hott-logic/book-labelnumbers-first-edition.tex, source lines 1324–1324. The transport rule for arrow types applied at source line 197. Navigation pointer only: the rule is recalled in the retained text itself and nothing of the counted argument is delegated to it. Its defining chapter is not part of the pinned corpus, so the pointer span is the label line of this chapter's label-number dump.

Source reference. Original section 2.10: sources/hott-logic/book-labelnumbers-first-edition.tex, source lines 1631–1631. The section that identifies transport in the family A \mapsto A along ua(e) with the application of e, used at source line 206. Navigation pointer only: the retained proof states the identification and uses it in one step. Its defining chapter is not part of the pinned corpus, so the pointer span is the label line of the dump.

Source reference. Original remark 2.12.6: sources/hott-logic/book-labelnumbers-first-edition.tex, source lines 1600–1600. The remark that the two booleans are distinct, invoked at source line 214 for the case analysis proving the swap has no fixed point. Navigation pointer only. Its defining chapter is not part of the pinned corpus, so the pointer span is the label line of the dump.

Theorem 3.2.2. *It is not the case that for all $A : \mathcal{U}$ we have $\neg(\neg A) \to A$.*

Proof. Recall that $\neg A \equiv (A \to \mathbf{0})$. We also read "it is not the case that …" as the operator $\neg$. Thus, in order to prove this statement, it suffices to assume given some $f : \prod_{(A:\mathcal{U})}(\neg\neg A \to A)$ and construct an element of $\mathbf{0}$.

The idea of the following proof is to observe that f, like any function in type theory, is "continuous". By univalence, this implies that f is *natural* with respect to equivalences of types. From this, and a fixed-point-free autoequivalence, we will be able to extract a contradiction.

Let $e : \mathbf{2} \simeq \mathbf{2}$ be the equivalence defined by $e(1_{\mathbf{2}}) :\equiv 0_{\mathbf{2}}$ and $e(0_{\mathbf{2}}) :\equiv 1_{\mathbf{2}}$, as in original remark 3.1.9. Let $p : \mathbf{2} = \mathbf{2}$ be the path corresponding to e by

univalence, i.e. $p :\equiv \mathsf{ua}(e)$. Then we have $f(\mathbf{2}) : \neg\neg\mathbf{2} \to \mathbf{2}$ and

$$\mathsf{apd}_f(p) : \mathsf{transport}^{A\mapsto(\neg\neg A\to A)}(p, f(\mathbf{2})) = f(\mathbf{2}).$$

Hence, for any $u : \neg\neg\mathbf{2}$, we have

$$\mathsf{happly}(\mathsf{apd}_f(p), u) : \mathsf{transport}^{A\mapsto(\neg\neg A\to A)}(p, f(\mathbf{2}))(u) = f(\mathbf{2})(u).$$

Now by (2.9.4), transporting $f(\mathbf{2}) : \neg\neg\mathbf{2} \to \mathbf{2}$ along p in the type family $A \mapsto (\neg\neg A \to A)$ is equal to the function which transports its argument along p^{-1} in the type family $A \mapsto \neg\neg A$, applies $f(\mathbf{2})$, then transports the result along p in the type family $A \mapsto A$:

$$\mathsf{transport}^{A\mapsto(\neg\neg A\to A)}(p, f(\mathbf{2}))(u) = \mathsf{transport}^{A\mapsto A}(p, f(\mathbf{2})(\mathsf{transport}^{A\mapsto\neg\neg A}(p^{-1}, u))).$$

However, any two points $u, v : \neg\neg\mathbf{2}$ are equal by function extensionality, since for any $x : \neg\mathbf{2}$ we have $u(x) : \mathbf{0}$ and thus we can derive any conclusion, in particular $u(x) = v(x)$. Thus, we have $\mathsf{transport}^{A\mapsto\neg\neg A}(p^{-1}, u) = u$, and so from $\mathsf{happly}(\mathsf{apd}_f(p), u)$ we obtain an equality

$$\mathsf{transport}^{A\mapsto A}(p, f(\mathbf{2})(u)) = f(\mathbf{2})(u).$$

Finally, as discussed in original section 2.10, transporting in the type family $A \mapsto A$ along the path $p \equiv \mathsf{ua}(e)$ is equivalent to applying the equivalence e; thus we have

$$e(f(\mathbf{2})(u)) = f(\mathbf{2})(u). \tag{3.2.3}$$

However, we can also prove that

$$\prod_{x:\mathbf{2}} \neg(e(x) = x). \tag{3.2.4}$$

This follows from a case analysis on x: both cases are immediate from the definition of e and the fact that $0_{\mathbf{2}} \neq 1_{\mathbf{2}}$ (original remark 2.12.6). Thus, applying (3.2.4) to $f(\mathbf{2})(u)$ and (3.2.3), we obtain an element of $\mathbf{0}$. □